

1. Introduction to Database Management Systems

A Database Management System (DBMS) is software that stores, organises and retrieves data while controlling concurrent access, security and recovery. Compared with a traditional file-processing system, a DBMS reduces data redundancy and inconsistency, enforces integrity constraints, supports concurrent users safely and provides backup and recovery.

Three-schema architecture: the external level (user views), the conceptual level (logical structure of the whole database) and the internal level (physical storage). The mappings between levels provide data independence.

Logical data independence is the ability to change the conceptual schema without changing external schemas or application programs. Physical data independence is the ability to change the internal schema (for example file organisation or indexes) without changing the conceptual schema.

2. Normalization

Normalization is the process of organising the attributes and tables of a relational database to minimise redundancy and to eliminate insertion, update and deletion anomalies. It works by decomposing a relation into smaller relations using functional dependencies, while keeping the decomposition lossless.

- Insertion anomaly: a fact cannot be stored without storing an unrelated fact.
- Update anomaly: the same fact is stored in many rows, so an update must change all of them.
- Deletion anomaly: deleting one fact unintentionally deletes another.

First Normal Form (1NF): every attribute holds only atomic (indivisible) values and there are no repeating groups.

Second Normal Form (2NF): the relation is in 1NF and no non-prime attribute is partially dependent on any candidate key (no partial dependency).

Third Normal Form (3NF): the relation is in 2NF and there is no transitive dependency of a non-prime attribute on a candidate key. Formally, for every functional dependency $X \rightarrow A$, either X is a superkey or A is a prime attribute.

Boyce-Codd Normal Form (BCNF): for every non-trivial functional dependency $X \rightarrow A$, X must be a superkey. BCNF is stricter than 3NF, and every BCNF relation is also in 3NF.

3. Functional Dependencies and Keys

A functional dependency $X \rightarrow Y$ holds on a relation R if any two tuples that agree on the attributes X also agree on the attributes Y .

Armstrong's axioms are sound and complete inference rules for functional dependencies:

- Reflexivity: if Y is a subset of X , then $X \rightarrow Y$.
- Augmentation: if $X \rightarrow Y$, then $XZ \rightarrow YZ$.
- Transitivity: if $X \rightarrow Y$ and $Y \rightarrow Z$, then $X \rightarrow Z$.

The closure of an attribute set X , written X^+ , is the set of all attributes functionally determined by X . If X^+ contains every attribute of R , then X is a superkey.

A candidate key is a minimal superkey. The primary key is the candidate key chosen by the designer to identify tuples. A foreign key is an attribute set in one relation that refers to the primary key of another relation and enforces referential integrity.

4. Indexing and B-Trees

An index is an auxiliary structure that speeds up data retrieval at the cost of extra storage and slower writes.

A B-tree of order m is a self-balancing search tree in which every node has at most m children, every internal node except the root has at least $\lceil m/2 \rceil$ children, and all leaves appear at the same level. Keys inside a node are kept in sorted order.

Advantages of B-trees:

- The tree always stays balanced, so search, insertion and deletion take $O(\log n)$ time.
- Each node holds many keys (high fan-out), so the tree is shallow and the number of disk I/O operations is small. This makes B-trees ideal for databases and file systems stored on disk.
- Nodes are kept at least half full, giving good storage utilisation.

A B+ tree stores all records (or record pointers) only in the leaf nodes, and the leaves are linked together. Internal nodes contain only keys used for routing. Because of the linked leaves, B+ trees are very efficient for range queries and sequential scans, which is why most relational databases use B+ trees for their indexes.

5. Transactions and Concurrency Control

A transaction is a logical unit of work that must satisfy the ACID properties:

- Atomicity: either all operations of the transaction happen or none do.
- Consistency: a transaction takes the database from one consistent state to another.
- Isolation: concurrent transactions do not see each other's intermediate results.
- Durability: once committed, changes survive system failures.

A schedule is serializable if its effect is equivalent to some serial execution of the same transactions.

Conflict serializability can be tested with a precedence graph: the schedule is conflict serializable if and only if the graph has no cycle.

Two-Phase Locking (2PL) guarantees conflict serializability. In the growing phase a transaction may acquire locks but not release any; in the shrinking phase it may release locks but not acquire new ones.

Strict 2PL holds all exclusive locks until commit, which also avoids cascading rollbacks. 2PL can still lead to deadlock, which is handled by detection (wait-for graph) or prevention (wait-die, wound-wait).